21 Days Python Report

PROJECT

mini_payment_program.py

```
import csv
import os
from datetime import datetime
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet

class Expense:
    def __init__(self, amount, category, date, description):
        self.amount = amount
        self.category = category
        self.date = date
        self.description = description


class ExpenseTracker:
    def __init__(self):
        self.expenses = []
        self.budget = 0
        self.load_data()

    def add_expense(self):
        try:
            amount = float(input("Enter amount: "))
            category = input("Enter category: ")
            description = input("Enter description: ")
            date = datetime.now().strftime("%Y-%m-%d")

            exp = Expense(amount, category, date, description)
            self.expenses.append(exp)

            print("Expense added!")

        except ValueError:
            print(" Invalid input!")

    def view_expenses(self):
        if not self.expenses:
            print("No expenses found.")
            return

        print("\n--- Expenses ---")
        for e in self.expenses:
            print(f"{e.date} | {e.category} | â‚¹{e.amount} | {e.description}")

    def total_spent(self):
        total = sum(e.amount for e in self.expenses)
        print(f"ðŸ’° Total Spent: â‚¹{total}")

        if self.budget:
            print(f"ðŸ“Š Budget: â‚¹{self.budget}")
            print(f"Remaining: â‚¹{self.budget - total}")

    def set_budget(self):
        try:
            self.budget = float(input("Enter budget: "))
            print(" Budget set!")
        except ValueError:
            print(" Invalid input!")

    def save_data(self):
        with open("expenses.csv", "w", newline="") as file:
            writer = csv.writer(file)
            writer.writerow(["Amount", "Category", "Date", "Description"])

            for e in self.expenses:
                writer.writerow([e.amount, e.category, e.date, e.description])

        print("Data saved to CSV!")

    def load_data(self):
```

```
    def load_data(self):
        if not os.path.exists("expenses.csv"):
            return

        with open("expenses.csv", "r") as file:
            reader = csv.reader(file)
            next(reader)  # skip header

            for row in reader:
                amount, category, date, description = row
                self.expenses.append(
                    Expense(float(amount), category, date, description)
                )

    def export_pdf(self):
        if not self.expenses:
            print("No data to export.")
            return

        doc = SimpleDocTemplate("expense_report.pdf")
        elements = []
        styles = getSampleStyleSheet()

        # Title
        elements.append(Paragraph("Expense Report", styles["Title"]))

        # Table Data
        data = [["Date", "Category", "Amount", "Description"]]

        for e in self.expenses:
            data.append([e.date, e.category, f"â,¹{e.amount}", e.description])

        table = Table(data)

        table.setStyle(TableStyle([
            ("BACKGROUND", (0, 0), (-1, 0), colors.grey),
            ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
            ("GRID", (0, 0), (-1, -1), 1, colors.black)
        ]))

        elements.append(table)

        doc.build(elements)

        print(" PDF exported successfully!")


def main():
    tracker = ExpenseTracker()

    while True:
        print("\n===== Expense Tracker =====")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Total Spending")
        print("4. Set Budget")
        print("5. Save to CSV")
        print("6. Export to PDF")
        print("7. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            tracker.add_expense()

        elif choice == "2":
            tracker.view_expenses()

        elif choice == "3":
            tracker.total_spent()

        elif choice == "4":
            tracker.set_budget()

        elif choice == "5":
            tracker.save_data()

        elif choice == "6":
            tracker.export_pdf()

        elif choice == "7":
            tracker.save_data()
```

```
            print("Exiting...")
            break

        else:
            print("â Œ Invalid choice!")


if __name__ == "__main__":
    main()
```

DAY1

main.py

```
import utitlity as ut

a=int(input("Enter a number"))
x=int(input("Enter a number to calculate GCD and LCM"))
y=int(input("Enter a number to calculate GCD and LCM"))


print(ut.factorial(a))
print(ut.GCD(x,y))
print(ut.lcm(x,y))
print(ut.perfectnumber(a))
print(ut.isprime(a))
```

utitlity.py

```
# This function or part of program will contain the utility functions which we will call in the main prc

def isprime(n):
    c=0
    for i in range(1,n):
        if n%i==0:
            c+=0
    if c==1:
        print(f"The number {n} is prime")
    else:
        print(f"The number {n} is not prime")


def factorial(n):
    fact=1
    for i in range(1,n+1):
        fact*=i
    print(f"The factorial of the number {n} is {fact}")


def GCD(x,y):
    if x==0:
        return y
    return GCD(y%x ,x)


def lcm(x,y):
    def c_gcd(x,y):
        while(y):
            x,y=y,x%y
        return x
    def c_lcm(x,y):
        lc=(x*y)//c_gcd(x,y)
        return lc
    print(f"The LCM of {x} and {y} is {c_lcm(x,y)}")


def perfectnumber(n):
    sum=0
    for i in range(1,n):
        if n%i==0:
            sum+=i
    if n==sum:

        print(f"The number {n} is a Perfect number")
    else:
        print(f"The number {n} is a Perfect number")
```

DAY10

book.py

```
class Book:
    library_name="Badola Library"

    def __init__(self,author,title,price):
        self.title=title
        self.author=author
        self.__price=price   #__(double underscore) means private variable

    def display(self):
        print(f"Book {self.title} by  {self.author}")

    def get_price(self):
        return self.__price


book1=Book("Mohit","Python Basics",300)
book2=Book("Rishabh","ALL ABOUT ITCS",1500)

book1.display()
print("Price:",book1.get_price())

book2.display()
print("Library:",Book.library_name)
```

payment.py

```
from abc import ABC,abstractmethod
class payment(ABC):
    @abstractmethod
    def pay(self,amount):
        pass


class cardpayment(payment):
    def pay(self,amount):
        return f"Paid Rs.{amount} using card"


class UPI(payment):
    def pay(self,amount):
        return f"Paid Rs.{amount} using UPI"

class bill :
    def __init__(self,amount):
        self.amount=amount

    def __str__(self):
        return f"Bill Amount: rs.{self.amount}"

    def __add__(self,other):                        #Function Overloading +
        return bill(self.amount +other.amount)

b1=bill(523)
b2=bill(3548)

print(b1)

b3=b1+b2
print(f"Total : {b3}")

payment1=cardpayment()
payment2=UPI()

print(b3)
print(payment1.pay(b2.amount))
print(payment2.pay(b1.amount))
```

DAY11

mergesort.py

```
def mergesort(arr,l,m,r):
    n1=m-l+1
    n2=r-m

    L=[0]*n1
    R=[0]*n2

    for i in range(n1):

        L[i]=arr[l+i]
    for j in range(n2):
        R[j]=arr[m+1+j]

    i=j=0

    k=l

    while i
```

Sorting all types.ipynb

ALL TYPES OF SORT

Bubble Sort

```
def bubblesort(arr):
    n=len(arr)
    for i in range(n):
        swapped=False
        for j in range(0,n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
                swapped=True

        if not swapped:
            break

arr=[32,42,63,62,62,63,63,5,2]
bubblesort(arr)
print(arr)
```

```
[2, 5, 32, 42, 62, 62, 63, 63, 63]
```

Selection Sort

```
def selectionsort(arr):
    n=len(arr)
    for i in range(n-1):
        min=i
        for j in range(i+1,n):
            if arr[j]<arr[min]:
                min=j
        arr[i],arr[min]=arr[min],arr[i]

arr=[32,42,63,62,62,63,63,5,2]
selectionsort(arr)
print(arr)
```

```
[2, 5, 32, 42, 62, 62, 63, 63, 63]
```

Insertion Sort

```
def insertionsort(arr):
    n=len(arr)
    if n<=1:
        return

    for i in range(1,n):
        key = arr[i]
        j=i-1
        while j>=0 and key < arr[j]:
            arr[j+1]=arr[j]
            j-=1
        arr[j+1]=key
arr=[32,42,63,62,62,63,63,5,2]
insertionsort(arr)
print(arr)
```

```
[2, 5, 32, 42, 62, 62, 63, 63, 63]
```

Merge Sort (Divide & Conquer)

```
def mergesort(arr,l,m,r):
    n1=m-l+1
    n2=r-m

    L=[0]*n1
    R=[0]*n2

    for i in range(n1):

        L[i]=arr[l+i]
    for j in range(n2):
        R[j]=arr[m+1+j]

    i=j=0

    k=l

    while i<n1 and j <n2:
        if L[i]<=R[j]:
            arr[k]=L[i]
            i+=1

        else:
            arr[k]=R[j]
            j+=1

        k+=1
    while i<n1:
        arr[k] =L[i]
        i+=1
        k+=1
    while j<n2:
        arr[k] =R[j]
        j+=1
        k+=1


def mergesoting(arr,l,r):
    if l<r:

        m=(l+r)//2
        mergesoting(arr,l,m)
        mergesoting(arr,m+1,r)
        mergesort(arr,l,m,r)

arr=[32,42,63,62,62,63,63,5,2]
print(f"Given Arraya{arr}")

mergesoting(arr,0,len(arr)-1)

print(f"Sorted array is {arr}")
```

```
Given Arraya[32, 42, 63, 62, 62, 63, 63, 5, 2]
Sorted array is [2, 5, 32, 42, 62, 62, 63, 63, 63]
```

Quick Sort

DAY12

vehical_system.py

```
 class vehical:
     def __init__(self,name):
         self.name=name

     def start(self):
         print(f"{self.name} is starting ")

     def fueltype(self):
         print("Generic Fuel")

 class car(vehical):
     def __init__(self,name,brand):
         super().__init__(name)
         self.brand=brand

     def fueltype(self):
         print(f"{self.name }runs on Petrol/Diesel")

     def start(self):
         print(f"{self.brand} car start with a key or button")
```

```
        print(f"{self.brand} car start with a key or button")

class bike(vehical):
    def __init__(self,name,cc):
        super().__init__(name)
        self.cc=cc

    def fueltype(self):
        print(f"{self.name }runs on Petrol")

    def start(self):
        print(f"{self.name} car start with self-start/kick")


vehicles=[
    car("car1","Toyota"),
    bike("bike1",200)
]

for v in vehicles:
    v.start()
    v.fueltype()
```

DAY13

Pandas.ipynb

----------PANDAS----------

What is Numpy?

It aims to provide an array object that is up to 50x faster than the traditional python lis

Why array is faster than the usual list?

Beacuse they are stored in the one continious memory unlike list, so the process to access

Normal / 1D Array

```
import numpy

arr=numpy.array([1,2,3,4])
arr
```

```
array([1, 2, 3, 4])
```

```
import numpy as np
arr=np.array([1,2,3,4])
arr
```

```
array([1, 2, 3, 4])
```

2D array

```
arr1=np.array([[1,2,3],[4,5,6]])
arr1
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
print(arr1)
```

```
[[1 2 3]
 [4 5 6]]
```

Custom / Special Creation function

```
# Array with all zeroes
arr2=np.zeros(5)
arr2
```

```
array([0., 0., 0., 0., 0.])
```

```
# 2X3 matrix of 1
arr3=np.ones((2,3))
arr3
arr4=np.ones((10,10))
arr4
```

```
array([[1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

Range creation

```
 np.arange(1,10)
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
 np.linspace(1,10,5) #5 values between 0 and 1
 #np.linespace(from,to,numberofelements)
```

```
array([ 1.  ,  3.25,  5.5 ,  7.75, 10.  ])
```

Accessing elements and modifying them as like list

```
 arr=np.array([10,20,30])
```

```
 print(arr[1])
```

```
20
```

```
arr[1]=73
```

```
print(arr[1])
```

```
73
```

```
arr[1][2]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[29], line 1
----> 1 arr[1][2]

IndexError: invalid index to scalar variable.
```

Deleting an elemnt in the array

```
arr=np.array([1,2,3,4])
# index 0=1,1=2,2=3,3=4
```

```
new_arr=np.delete(arr,2)
```

```
print(new_arr)
```

```
[1 2 4]
```

```
arr=np.array([1,2,3,4])
# 1+5,2+5,3+5,4+5
```

```
print(arr+5)
```

```
[6 7 8 9]
```

```
print(arr*5)
```

```
[ 5 10 15 20]
```

```
print(arr-25)
```

```
[-24 -23 -22 -21]
```

```
arr1=([1,2,3])
```

```
arr1=([1,2,3])
arr2=([4,5,6])
```

```
arr1+arr2
```

```
[1, 2, 3, 4, 5, 6]
```

DAY14

csvfile.ipynb

BASICS OF CSV FILE

```
Reading into csv file
```

```
 import csv
 with open ("Data.txt", "r") as f:
     reader=csv.reader(f)
     for row in reader:
         print(row)
```

```
['hi my name is Mohit', '']
['hi', 'my', 'name', 'is', 'mohit']
```

```
writing in csv file
```

```
 import csv
 with open ("Data.txt", "w") as f:
     writer=csv.writer(f)
     writer.writerow(["Welccome","everyone"])
     writer.writerow([["Welccome","everyone"],[25,30]])
```

```
 import csv
 with open ("Data.txt", "r") as f:
     reader=csv.reader(f)
     for row in reader:
         print(row)
```

```
['Welccome', 'everyone']
[]
["['Welccome', 'everyone']", '[25, 30]']
[]
```

```
import csv
with open ("Data.txt", "w") as f:
    writer=csv.writer(f)
    writer.writerow(["Welccome","everyone"])
    writer.writerows([["Welccome","everyone"],[25,30]])
```

```
import csv
with open ("Data.txt", "r") as f:
    reader=csv.reader(f)
    for row in reader:
        print(row)
```

```
['Welccome', 'everyone']
[]
['Welccome', 'everyone']
[]
['25', '30']
[]
```

Appending into a csv file

```
import csv
with open ("Data.txt", "a") as f:
    writer=csv.writer(f)
    writer.writerow(["Have","a","Goodday"])
    # writer.writerow([["Welccome","everyone"],[25,30]])
```

```
import csv
with open ("Data.txt", "r") as f:
    reader=csv.reader(f)
    for row in reader:
        print(row)
```

```
['Have', 'a', 'Goodday']
[]
```

```
import csv
f=open("Data.txt","a+")
writer=csv.writer(f)
writer.writerow([1,2,3,4])
reader=csv.reader(f)
# f.seek(0)
print(f.tell())
for i in reader:
    print(i)
f.close()
```

```
40
```

DAY15

mini_exercise.py

```
import csv
import os

def main():
    filename="student.csv"
    try:
        f=open(filename, "w", newline="")
        writer=csv.writer(f)
        writer.writerows(
            [
                [1,"Mohit",85],
                [2,"Amit",90],
                [3,"Riya",78]
            ]
        )
        f.close()
        print("Data written Successfully")

        with open(filename,"a",newline="") as f:
            writer=csv.writer(f)
            writer.writerow([4,"Sneha",88])

        print("Data appended successfully")

        f=open(filename,"r")
        reader=csv.reader(f)
        for i in reader:
            print(i)

        f.close()

        print("\n File Methods demo")
        print(f"File Name : {f.name} ")
        print(f"File Closed : {f.closed} ")
        print(f"File Mode: {f.mode} ")
```

```
        print("Current working directory", os.getcwd())
        print("Files in Directory", os.listdir())


        try:
            f=open("nofile.csv","r")
        except FileNotFoundError as e:
            print("Exception Caught", e)

        try:
            num=int("abc")
            result=10/0
        except ValueError as e:
            print("Value error caught : ",e)
        except ZeroDivisionError as e:
            print("Zero Division error caught : ",e)
        except Exception as e:
            print("General Exception Caught : ",e)
        else:
            print("No exception Occured")
        finally:
            print("finally bolck always executed")


        marks = 40
        if marks<50:
            raise Exception ("Marks too low")

        assert marks >=0, "Marks Cannot be negative"
    except Exception as e:
        print("Outer exception", e)

if __name__=="__main__":
    main()
```

DAY16

Array.ipynb

Numpy /Pandas

Reshape an array

```
import numpy as np
```

```
arr=np.array([1,2,3,4,5,6])

print(arr.reshape(2,3))
```

```
[[1 2 3]
 [4 5 6]]
```

```
arr=np.array([1,2,3,4,5,6])

print(arr.reshape(3,2))
```

```
[[1 2]
 [3 4]
 [5 6]]
```

```
arr=np.array([1,2,3,4,5,6])

print(arr.reshape(6,1))
```

```
[[1]
 [2]
 [3]
 [4]
 [5]
 [6]]
```

```
arr=np.array([1,2,3,4,5,6])

print(arr.reshape(1,6))
```

```
[[1 2 3 4 5 6]]
```

```
 arr=np.array([1,2,3,4,5,6])

 print(arr.reshape(4,1))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[7], line 3
      1 arr=np.array([1,2,3,4,5,6])
----> 3 print(arr.reshape(4,1))

ValueError: cannot reshape array of size 6 into shape (4,1)
```

Important Functions

```
 arr=np.array([1,2,3,4,5,6])
```

```
 print(np.sum(arr))
 print(np.mean(arr))
 print(np.std(arr))
 print(np.amax(arr))
```

```
21
3.5
1.707825127659933
6
```

```
 print(np.random.rand(3))
 print(np.random.randint(1,10,5))
```

```
[0.16366388 0.80993467 0.73212314]
[6 5 8 6 7]
```

Attributes

```
 arr=np.array([[1,2,3],[4,5,6]])

 print(arr.shape)
 print(arr.ndim)
 print(arr.size)
```

```
(2, 3)
2
6
```

Pandas Basics

```
 import pandas as pd
```

```
 s=pd.Series([10,20,30,40,50,60])
 print(s)
```

```
0    10
1    20
2    30
3    40
4    50
5    60
dtype: int64
```

```
 print(s.head())
 print(s.tail())
 print(s+5)
```

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
1    20
2    30
3    40
4    50
5    60
dtype: int64
0    15
1    25
2    35
3    45
4    55
5    65
dtype: int64
```

Data Frame Creation

```
 df=pd.DataFrame({
     "Name": ["A","B"],
     "Marks" : [90,80]
 })

 print(df)
```

```
  Name  Marks
0    A     90
1    B     80
```

Column Operations

```
print(df["Marks"])
```

```
0    90
1    80
Name: Marks, dtype: int64
```

```
df["Age"]=[20,22]
df
```

	Name	Marks	Age
0	A	90	20
1	B	80	22

```
df.drop("Age",axis=1, inplace=True)
df
```

	Name	Marks
0	A	90
1	B	80

Filtering

```
 print(df[df["Marks"]>85])
```

```
  Name  Marks
0    A     90
```

Missing Data

```
 df["Marks"][0]=None
 print(df.isnull())
 print(df.fillna(0))
 df
```

```
    Name  Marks
0  False  False
1  False  False
  Name  Marks
0    A     90
1    B     80

C:\Users\TUF A15\AppData\Local\Temp\ipykernel_17712\338736855.py:1: ChainedAssignmentError: A value is
being set on a copy of a DataFrame or Series through chained assignment.
Such chained assignment never works to update the original DataFrame or Series, because the intermedia
te object on which we are setting values always behaves as a copy (due to Copy-on-Write).

Try using '.loc[row_indexer, col_indexer] = value' instead, to perform the assignment in a single step
.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/us
er_guide/copy_on_write.html#chained-assignment
  df["Marks"][0]=None
```

	Name	Marks
0	A	90
1	B	80

Aggregation

```
df.to_csv("data.csv")
new_df=pd.read_csv("data.csv")
print(new_df)
```

```
   Unnamed: 0 Name  Marks
0           0    A     90
1           1    B     80
```

DAY17

morenumpy.ipynb

```
import numpy as np
import pandas as pd
```

1. Row-wise & Column-wise Sum

```
arr=np.array([[1,2,3],[4,5,6]])
print("Row Sum :", np.sum(arr,axis=1) )
print("Column Sum :", np.sum(arr,axis=0) )
```

```
Row Sum : [ 6 15]
Column Sum : [5 7 9]
```

2. Find Max Element and Its Index

```
arr=np.array([10,50,20,80,30])

print("Max : ", np.max(arr))
print("Index of Max : ", np.argmax(arr))
```

```
Max :  80
Index of Max :  3
```

3. Matrix Multiplication

```
a=np.array([[1,2],[3,4]])
b=np.array([[5,6],[7,8]])
# c=np.array([[15,6],[7,8]])

print("Matrix Multiplication:", np.dot(a,b))
```

```
Matrix Multiplication: [[19 22]
 [43 50]]
```

4. Replace Elements Based on Condition

```
 arr=np.array([1,7,3,4,5,6])

 arr[arr>3]=0
 print(arr)
```

```
[1 0 3 0 0 0]
```

Pandas

1. Student Data Analysis

```
 df=pd.DataFrame({
     "Name":["A","B","C","D"],
     "Marks":[90,75,60,85]
 })

 print("Average :",df["Marks"].mean())
 print("Max Marks :",df["Marks"].max())

 print("Students > 80: \n", df[df["Marks"]>80])
```

```
Average : 77.5
Max Marks : 90
Students > 80:
   Name  Marks
0    A     90
3    D     85
```

Adding Grade column

```
df["Grade"]=["A" if m>80 else "B" for m in df["Marks"]]
```

```
df
```

	Name	Marks	Grade
0	A	90	A
1	B	75	B
2	C	60	B
3	D	85	A

Renaming the column Marks to Score and deleting the Grade column drop

```
df.drop("Grade",axis=1,inplace=True)
df.rename(columns={"Marks":"Score"}, inplace=True)
df
```

	Name	Score
0	A	90
1	B	75
2	C	60
3	D	85

DAY18

classes_and_obj_with_numpy_pandas.ipynb

NumPy + Pandas combined with Classes, Objects & some magic methods (r

1. NumPy inside Class (Array Operations)

```
import numpy as np
```

```
class Arrayanalyser:
    def __init__(self,data):
        self.arr=np.array(data)

    def show(self):
        print("Array : ",self.arr)

    def stats(self):
        print("Sum : ", np.sum(self.arr))
        print("Mean : ", np.mean(self.arr))
        print("Std : ", np.std(self.arr))
```

```
a=Arrayanalyser([1,2,3,4,5,6])
```

```
a.show()
a.stats()
# Arrayanalyser([1,2,3,4,5,6]).stats  # need magic function for this
```

```
Array :  [1 2 3 4 5 6]
Sum :  21
Mean :  3.5
Std :  1.707825127659933

<bound method Arrayanalyser.stats of <__main__.Arrayanalyser object at 0x000001AA519FDBD0>>
```

2. Using **str** with NumPy

```
class arraydisplay:
    def __init__(self,data):
        self.arr=np.array(data)

    def __str__(self):
        return f"My array is : {self.arr}"
```

```
a=arraydisplay([10,20,30,40])
print(a)
```

```
My array is : [10 20 30 40]
```

3. Overloading + for Arrays

```
class arrayadd:
    def __init__(self,data):
        self.arr=np.array(data)

    def __add__(self,other):
        return self.arr+ other.arr
```

```
a1=arrayadd([1,2,3])
a2=arrayadd([1,2,3])

print(a1+a2)
```

```
[2 4 6]
```

4. Pandas DataFrame in Class

Q4. Student Data Manager

```
import pandas as pd
```

```
class studentdata:
    def __init__(self):
        self.df=pd.DataFrame({
            "Name" : ['A','B','C'],
            "Marks" : [90,75,60]

        })

    def show(self):
        print(self.df)

    def average(self):
        print(f"Average : ",self.df["Marks"].mean())

    def topper(self):
        print(self.df[self.df["Marks"]==self.df["Marks"].max()])
```

```
s=studentdata()
s.show()
s.average()
s.topper()
```

```
  Name  Marks
0    A     90
1    B     75
2    C     60
Average :  75.0
  Name  Marks
0    A     90
```

OPERATOR OVERLOADING AND MORE ON OVERLOADING FUNCTIONS

DAY19

overloading.ipynb

Function Overloading

Same function name, different behavior depending on arguments

```
class Mathsops:
    def add(self,a,b=5,c=0):
        return a+b+c
a=Mathsops()
print(a.add(3,7))
print(a.add(3,6,3))
print(a.add(3))
```

```
10
12
8
```

Using *args Any number of arguments

```
class Mathsops:
    def add(self,*args):
        return sum(args)
a=Mathsops()
print(a.add(3,7))
print(a.add(3,6,3))
print(a.add(3,3,3234,6,4,3,3,4,5,6))
```

```
10
12
3271
```

Operator Overloading

Giving new meaning to operators like +, -, *

```
class Number:
    def __init__(self,value):
        self.value=value

    def __add__(self,other):
        return self.value+other.value


n1=Number(10)
n2=Number(56)
print(type(n1))
print(n1+n2)
```

```
<class '__main__.Number'>
66
```

overloading with multiple objects

```
class Number:
    def __init__(self,value):
        self.value=value

    def __add__(self,other):
        return Number(self.value+other.value)

    def __str__(self):
        return str(self.value)


n1=Number(10)
n2=Number(56)
n3=Number(562)
n4=Number(562)

print(n1+n2+n3+n4)
```

```
1190
```

Operator Overloading for Subraction

```
class Number:
    def __init__(self,value):
        self.value=value

    def   sub  (self,other):
```

```
        return Number(self.value-other.value)

    def __str__(self):
        return str(self.value)


n1=Number(10)
n2=Number(56)
n3=Number(52)
n4=Number(2)

print(n1-n2-n3-n4)
```

```
-100
```

Operator Overloading for Multiplication

```
class Number:
    def __init__(self,value):
        self.value=value

    def __mul__(self,other):
        return Number(self.value*other.value)

    def __str__(self):
        return str(self.value)


n1=Number(10)
n2=Number(56)
n3=Number(562)
n4=Number(562)

print(n1*n2*n3*n4)
```

```
176872640
```

While using all the operator overloading together python F0llows BODMAS

DAY2

systemmodule.py

```
# Question 1: System Information Tool (sys module)
# Using sys module, create a program that displays:
# â€¢ Python version (sys.version)
# â€¢ Platform information (sys.platform)
# â€¢ Maximum integer size (sys.maxsize)
# â€¢ Command line arguments (sys.argv) - create a calculator that takes numbers as
# command line arguments
# â€¢ Python path (sys.path)
# Format the output in a professional report style with clear sections and headers.

import sys

def sysinfo():
    print("System Information")


    print("Version")
    print(sys.version)

    print("Platform")
    print(sys.version)

    print("MaxSize Integer")
    print(sys.maxsize)


    print("Path")
    for i in sys.path:
        print("->",i)

def calculator():
    if len(sys.argv) < 4:
        print("Enter the value in value operator value manner")
    num1=float(sys.argv[1])
    num2=float(sys.argv[3])
    operator=(sys.argv[2])   # suppose input [systemodule.py,4,+,4]

    if operator=="+":
        print(f"{num1} {operator} {num2} = {num1+num2}")
    if operator=="-":
        print(f"{num1} {operator} {num2} = {num1-num2}")
    if operator=="*":
        print(f"{num1} {operator} {num2} = {num1*num2}")
    if operator=="/":
        print(f"{num1} {operator} {num2} = {num1/num2}")

    else:
        print("Invalid Chooice")

sysinfo()
calculator()
```

DAY20

abstract.ipynb

Creating an Abstract Base Class

importing abstract as abc

ABC -> Abstract Base Class

used to declare abstract methods

Abstract method it foreces child classes to implement its method

it has no implementation the function below pay()

```
from abc import ABC , abstractmethod

class payment(ABC):
    @abstractmethod
    def pay(self,amount):
        pass
```

Implementation

Inheritance and overrriding the pay()

```
class UPI(payment):
    def pay(self,amount):
        return f"Paid {amount} via UPI"
class Card(payment):
    def pay(self,amount):
        return f"Paid {amount} via Card"
```

```
p=UPI()
```

```
p=UPI()
p2=Card()
```

```
print(p.pay(500))
print(p2.pay(5580))
```

```
Paid 500 via UPI
Paid 5580 via Card
```

Abstract Class with Concrete Methods

```
class payment(ABC):
    def receipt(self):  #Normal Function
        return "Recipt Graunted"

    @abstractmethod
    def pay(self,amount):
        pass
```

```
class UPI(payment):
    def pay(self,amount):
        return f"Paid {amount} via UPI"

p=UPI()
```

```
print(p.pay(400))
```

```
Paid 400 via UPI
```

```
print(p.receipt())
```

```
Recipt Graunted
```

Static Method and class object

class object refers to class itself it does not require self as a first parameter

uses @classmethod

self refers to the current object whreas cls refers to the class itself

cls is used in class method and self is used in instance method

self has objectvariable access and cls has class variable access

```
class student:
    school="XYZ School"

    def __init__(self,name):
        self.name=name

    def show_name(self): #Instance Method
        print(self.name)

    @classmethod
    def show_school(cls):  #Class Method
        print(cls.school)
```

```
s=student("Mohit")
```

```
s.show_name()
student.show_school()
```

```
Mohit
XYZ School
```

- It belongs to a class
- it soess nor use self object or cls object

```
class classname:
    @staticmethod
    def methodname(parameters):
        #code
        pass
```

```
class classname:
    @staticmethod
    def add(a,b):
        return a+b
```

```
print(classname.add(5,3))
```

```
8
```

DAY21

more_method.ipynb

Type	Keyword	Works On	Used For
Instance Method	self	Object	Object data
Class Method	cls	Class	Class data
Static Method	None	Independent	Utility

```
class Student:
    school = "ABC School"

    @classmethod
    def change_school(cls, new_school):
        cls.school = new_school
```

```
print(Student.school)
```

```
ABC School
```

```
Student.change_school("XYZ School")

print(Student.school)
```

```
XYZ School
```

```
 class student:
     school="XYZ School"  # Class Variable

     def __init__(self,name):
         self.name=name

     def show_name(self): #Instance Method
         print(self.name)

     @classmethod
     def show_school(cls):  #Class Method
         print(cls.school)
```

```
 student.school
```

```
'XYZ School'
```

```
 s1=student.show_name
```

```
 s1=student("Mohit")
 s2=student("Ravi")
```

```
 print(s1.school)
 print(s2.school)
```

```
XYZ School
XYZ School
```

Changing class variable using class

```
 student.school="ABC School"
```

```
 print(s1.school)
 print(s2.school)
```

```
ABC School
ABC School
```

Changing Class variable Using Class Method

```
 student.show_school()

 student.school="MNO School"

 student.show_school()
```

```
MNO School
MNO School
```

```
 s1.school="LMNO School"
```

```
 print(s1.school)
 print(s2.school)
```

```
 print(student.school)
```

```
LMNO School
MNO School
MNO School
```

Changing the Instance variable

```
 s1.name="Amamn"
```

```
 print(s1.name)
 print(s2.name)
```

```
Amamn
Ravi
```

```
 s1.show_name()
```

```
Amamn
```

BAse Code for static Method

```
class Student:
    school = "ABC School"   # class variable

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def show_name(self):   # Instance Method
        print(self.name)

    @classmethod
    def show_school(cls):  # Class Method
        print(cls.school)

    @staticmethod
    def is_pass(marks):    # Static Method
        return marks >= 40

    @staticmethod
    def change_school():
        Student.school="TDS School"

    @staticmethod
    def change_marks(obj,new_marks):
        obj.marks=new_marks
```

```
s1=Student("Mohit",80)
s2=Student("Ravi ",30)
```

```
print(Student.is_pass(80))
print(Student.is_pass(30))
```

```
True
False
```

Using Static Method with Object

```
print(s1.is_pass(s1.marks))
```

```
True
```

Modifying class variable

```
 Student.change_school()

 print(s1.school)
 print(s2.school)
```

```
TDS School
TDS School
```

```
 Student.change_marks(s1,95)

 print(s1.marks)
 print(s2.marks)
```

```
95
30
```

Static method neither depends on object nor class â€” it only works on the data you pass.â€

DAY3

time.py

```
 # Create programs using time module:
```

```
# â€¢ Display current time using time.time(), time.ctime(), time.localtime()
# â€¢ Format time using time.strftime() with different format codes
# â€¢ Create countdown timer using time.sleep()
# â€¢ Calculate age in years, months, days from birth date
# â€¢ Find day of week for any given date
# Display results in user-friendly format with proper labels

import time

print ("All the time formats")
current_time=time.time()
print(f"Current time = {current_time}")

print(f"C_time= {time.ctime(current_time)}")


print(f"Local time = {time.localtime(current_time)}")


print("Structured time")

current_time=time.localtime()
print(current_time)

print(f"Time in (dd-mm-yy) =", time.strftime("%d-%m-%y", current_time))

print("time in (HH-MM-SS)", time.strftime("%H-%M-%S", current_time))

print("Day of the week", time.strftime("%A", current_time))

print("Name of the Month", time.strftime("%B",current_time))

print("Time in full format \n\n",time.strftime("%A %d %B %H:%M:%S %p",current_time))
```

timer.py

```
import time


seconds=int(input("Enter the time in seconds"))

print("COUNTDOWN STARTS!!!!")

for i in range(seconds,0,-1):
    print(i)
    time.sleep(1)

print("Alert Times UPPPPP")
```

DAY4

__init__.py

DAY5

bank.py

```
class bankaccount:
    def __init__ (self, account_holder,balance=0):
        self.account_holder=account_holder
        self.balance=balance

    def deposit(self,amount):
        if amount>0:
            self.balance+=amount
            print(f"{amount} deposited sucessfully")
        else:
            print(f"Invalid deposit amount")


    def withdrew(self,amount):
        if amount>0:
            self.balance-=amount
            print(f"{amount } withdrewn sucessfully")
        else:
            print("Invalid Amount")
```

```
    def checkbalance(self):
        print(f"Balance : {self.balance}")


acc1=bankaccount("Mohit",10000)

acc1.deposit(500)
acc1.checkbalance()

acc1.withdrew(320)
acc1.checkbalance()
```

task_priority.py

```
class Taskmanager:
    def __init__(self):
        self.task=[]
    def add_task(self,task,priority):
        self.task.append({"task":task , "Priority":priority})
        print(f"Task '{task}' added to the priority '{priority}'")

    def show_task(self):
        if not self.task:
            print("No task available")
            return

        sorted_tasks = sorted(self.task, key=lambda x: x["Priority"])
        print("Tasks")

        for t in sorted_tasks:
            print(f"Task: {t['task']} | Priority: {t['Priority']}")

t=Taskmanager()
t.add_task("Comple Assignment",5)
t.add_task("Sleep",7)
# t.add_task("Eat Biryani",25)

t.show_task()
```

DAY6

limiter.py

```
# Create class RateLimiter:

# allow only N requests per minute
# block extra requests

# ðŸ'‰ Concept: time + system design (used in APIs ðŸ”¥)

# The main goal is to:

# Allow only a limited number of requests (say N requests)
# Within a time window of 1 minute (60 seconds)
# If the limit is exceeded â†’ block further requests


import time

class RateLimiter:
    def __init__(self,max_limits):
        self.max_limits=max_limits
        self.requests=[]

    def allow_request(self):
        current_time=time.time()
        self.requests=[t for t in self.requests if current_time-t < 60]
        if len(self.requests)
```

otp.py

```
import random
import time

class OTPSystem:
    def __init__(self):
        self.otp=None
        self.generated_time=None
```

```
    def generate_otp(self):
        self.otp=random.randint(100000,999999)
        self.generated_time=time.time()

        print(f"Generated OTP : {self.otp}")

    def varify_otp(self,user_otp):
        if self.otp is None:
            print("No OTP Generated")
            return
        current_time=time.time()

        if current_time-self.generated_time>30:
            print("OTP Expired")
            return

        if user_otp==self.otp:
            print("OTP Verified")

        else:
            print("Invalid OTP")


otp_system=OTPSystem()
otp_system.generate_otp()

user_input=int(input("Enter 6 digit otp "))

otp_system.varify_otp(user_input)
```

DAY7

expense_tracker.py

```
from datetime import datetime
from collections import defaultdict

class Expenses:
    def __init__(self, amount,category,date):
        self.amount=amount
        self.category=category
        self.date=datetime.strptime(date,"%Y-%m-%d")

class ExpenseTracker:
    def __init__(self):
        self.expenses=[]

    def add_expenses(self,amount,category,date):
        expense=Expenses(amount,category,date)
        self.expenses.append(expense)

    def monthly_report(self,year,month):
        total=0
        for exp in self.expenses:
            if exp.date.year==year and exp.date.month == month:
                total+= exp.amount
        return total


    def highest_spending_category(self):
        category_total=defaultdict(int)
        for exp in self.expenses:
            category_total[exp.category]+=exp.amount

        if not category_total:
            return None

        return max(category_total, key=category_total.get)

tracker=ExpenseTracker()

tracker.add_expenses(500,"Food","2026-04-01")
tracker.add_expenses(30,"Chips","2026-04-30")
tracker.add_expenses(750,"Travel","2026-03-25")
tracker.add_expenses(800,"Food","2026-03-21")

print("April Total : ", tracker.monthly_report(2026,4))
```

```
print("Top category : ",tracker.highest_spending_category())
```

DAY8

note.py

```
import csv
FILE="Notes.csv"

def add_notes():
    title=input("Title : ")
    content= input("Content : ")

    with open(FILE , "a", newline="") as f:
        writer=csv.writer(f)
        writer.writerow([title,content])

    print("Notes Added")

def view_note():
    try:
        with open(FILE,"r") as f:
            f.seek(0)
            reader=csv.reader(f)
            for i,row in enumerate(reader,start=1):
                print(i, row[0], "-", row[1])

    except:
        print("No notes Found")


def delete_note():
    notes=[]
    try:
        with open(FILE, "r") as f:
            reader=csv.reader(f)
            notes=list(reader)

    except:
        print("No Notes to delete")
        return

    view_note()
    n=int(input("ENter the note number to delete "))

    if 0<=n
```

DAY9

shoppingcart.py

```
# Create a class ShoppingCart that:

# Stores items (name + price)
# Allows adding items
# Supports:
# len(cart) â†’ number of items
# item in cart â†’ check if item exists
# cart1 + cart2 â†’ merge two carts
# print(cart) â†’ show cart nicely

class shoppingCart:
    def __init__(self,name):
        self.name=name
        self.item=[]

    def add_item(self,item,price):
        self.item.append((item,price))


    def __str__(self):
        return f"{self.name}'s Cart with {len(self.item)} items"

    def __repr__(self):
        return f"ShoppingCart(name={self.name}, items={self.item})"

    def __len__(self):
        return len(self.item)
```

```
        def __add__(self,other):
            new_cart=shoppingCart(self.name + " & " + other.name)

            new_cart.item=self.item + other.item

            return new_cart

        def __eq__(self,other):
            return len(self.item)==len(other.item)


        def __lt__(self,other):
            return len(self.item)
```